



## UNIDADE III

---

Projeto de Sistemas  
Orientado a Objetos

Prof. Me. Edson Moreno

# Projetos de dados e classes e projeto arquitetural

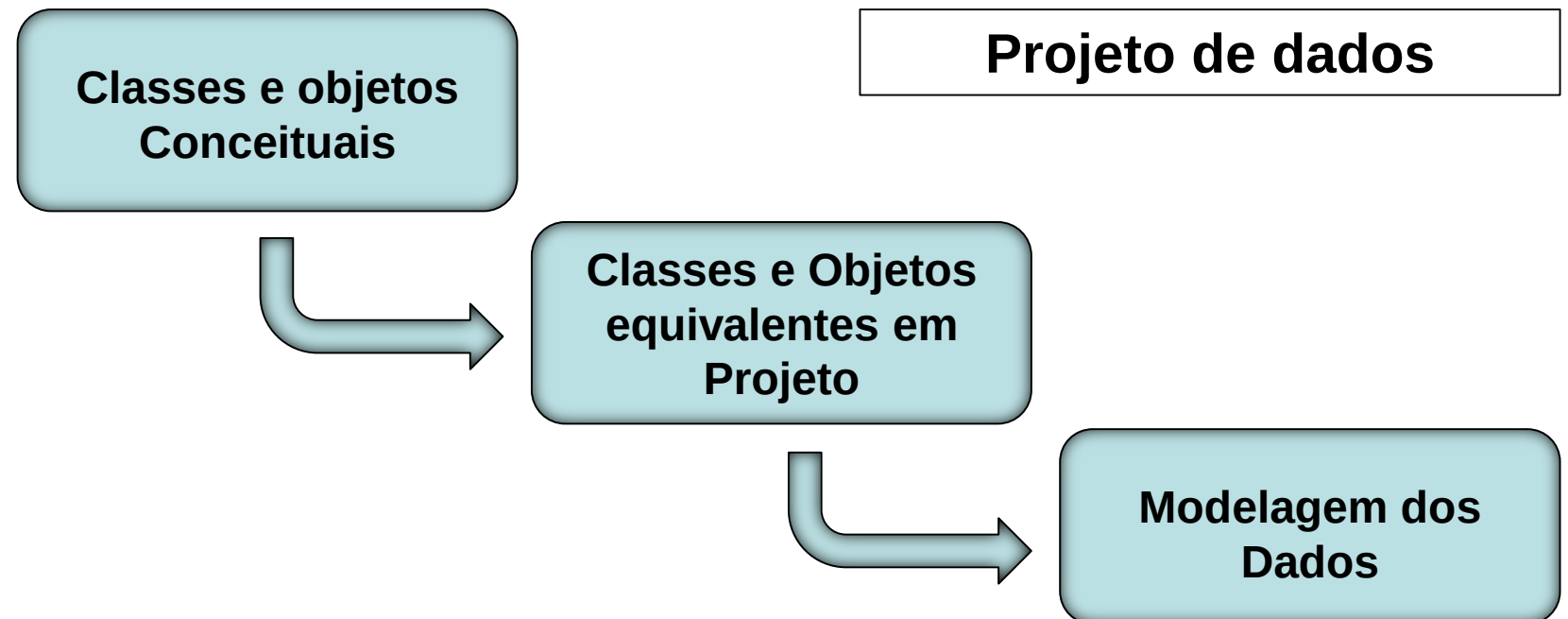
- A Unidade III se refere à fase de projeto dos dados, que tem o objetivo de definir um dicionário de dados, uma estrutura de informação necessária para a implementação de um sistema ou *software*.
- O projeto do banco segue os princípios do projeto de sistemas orientados a objeto, assunto abordado na Unidade I, ou seja, o projeto do banco de dados é dividido em três fases: projeto conceitual, projeto lógico e projeto físico.
- O modelo de dados relacional é um dos modelos mais utilizados em projetos de banco de dados, e, por isso, a ênfase é dada à produção do modelo entidade e relacionamento (MER). Na Unidade II foram abordados os aspectos da entidade-relacionamento (E-R).

Saiba mais:

SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. *Sistema de banco de dados*. 5. ed. Rio de Janeiro: Campus, 2006.

# Projeto de dados e classes

- A fase de projeto de dados e classes tem o foco nas tarefas técnicas do projeto, tendo como base o modelo de classes de projeto e dos aspectos dinâmicos do projeto de objetos mapeados até aqui.
- Segundo Pressman (2006), a fase de projetos de dados tem como objetivo a geração do modelo de dados e a transformação de classes e objetos conceituais em classes e objetos equivalentes em projeto.

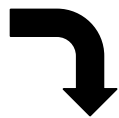


Fonte: Adaptado de: Moreno (2020).

# Introdução ao projeto de dados

- O projeto de dados, ou modelagem de dados, tem como objetivo definir uma estrutura de informações necessárias para implementar o sistema de *software* (PRESSMAN, 2006).
- Como foi visto, boa parte dessas atividades é feita pelo SGBD, que é o responsável por prover os mecanismos de acesso, gerenciamento e processamento das informações em um repositório de dados.
- Nesta fase a ênfase à organização das informações é em como representar um dicionário de dados que dê suporte à resolução do problema pelo sistema de *software*.

**Esta leitura vale a pena**



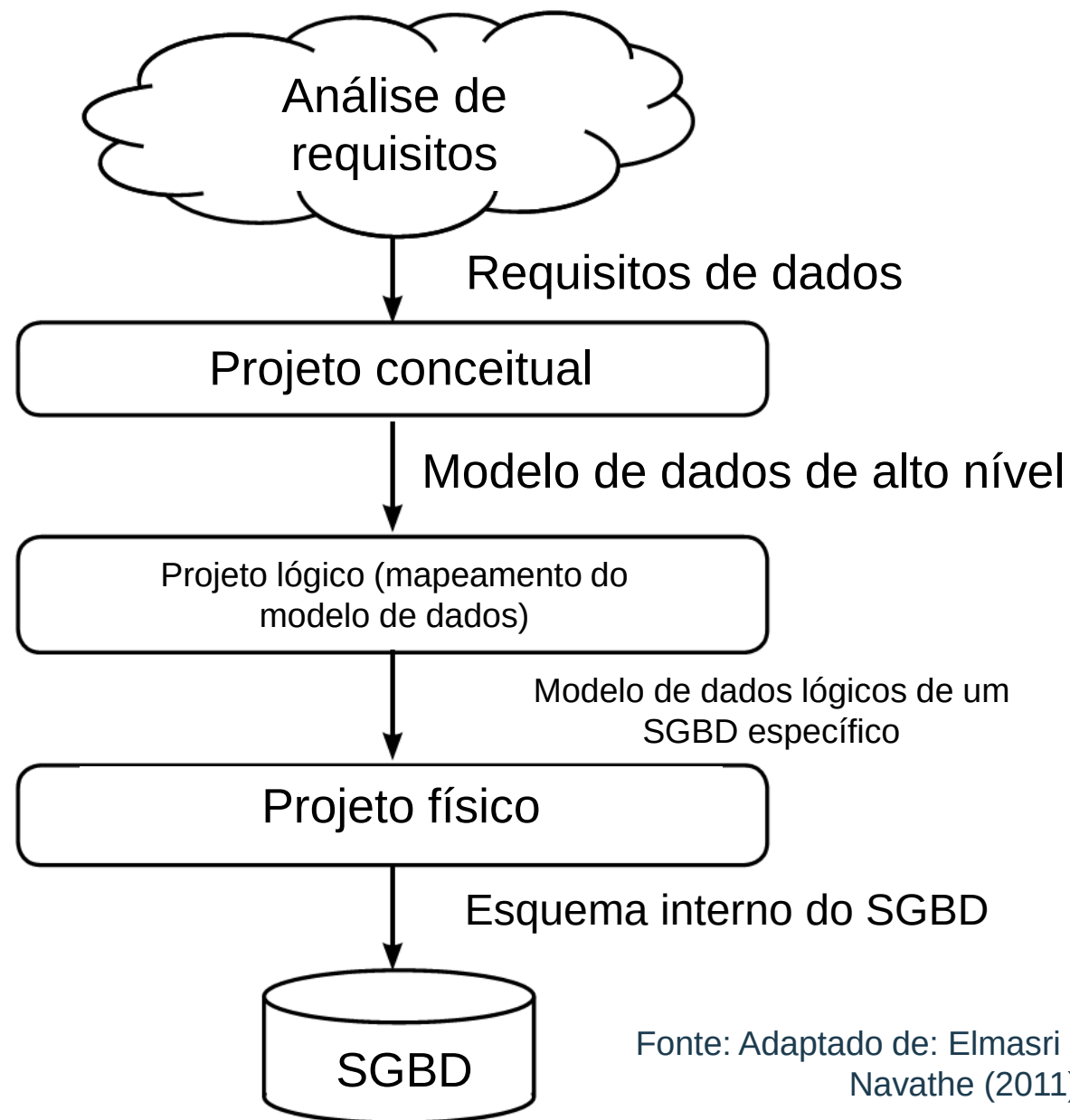
Saiba mais:

IBM. Data dictionary tables. IBM Knowledge Center.

Disponível em [https://www.ibm.com/support/knowledgecenter/pt-br/SSLKT6\\_7.6.0/com.ibm.mbs.doc/configur/r\\_data\\_dictionary\\_tables.html](https://www.ibm.com/support/knowledgecenter/pt-br/SSLKT6_7.6.0/com.ibm.mbs.doc/configur/r_data_dictionary_tables.html). Consultado em 23/06/2020.

# Introdução ao projeto de dados – Processo do projeto de banco de dados

O processo de projeto de banco de dados pode ser subdividido em três fases: projeto conceitual, projeto lógico e projeto físico, como mostra a figura a seguir:

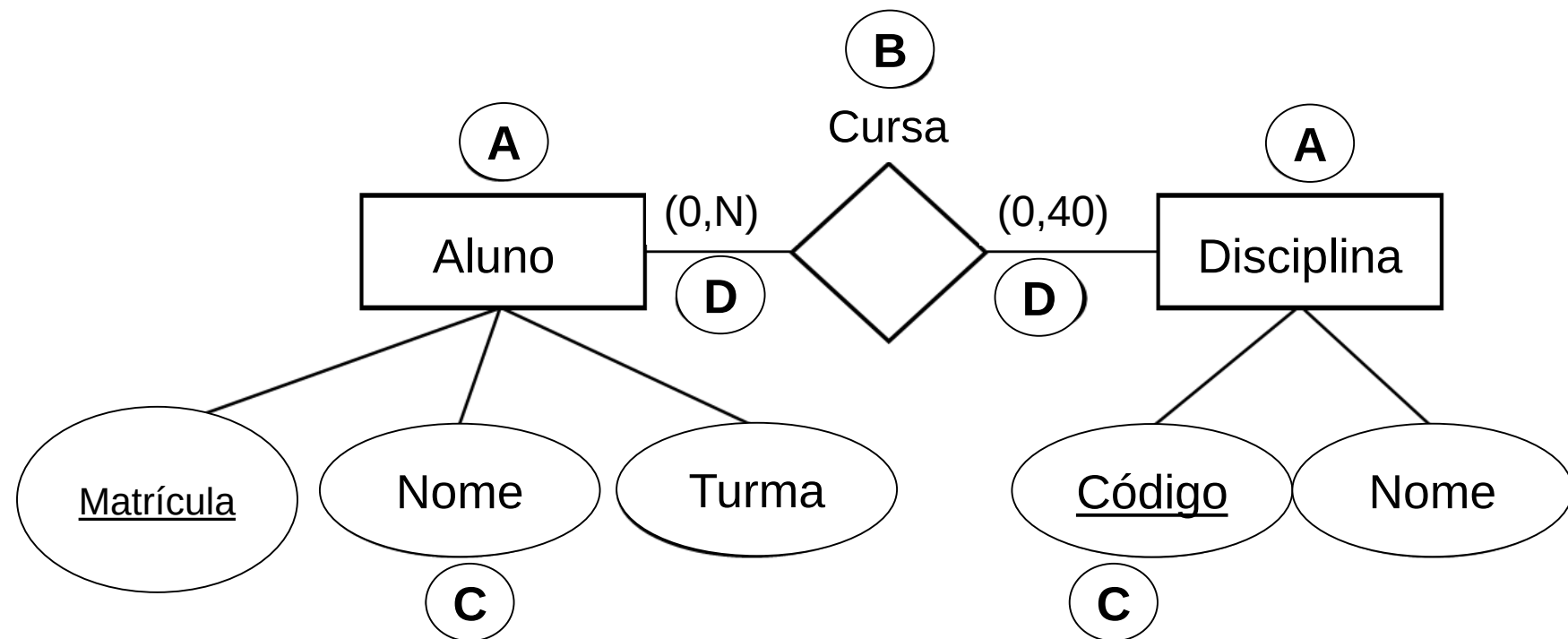


Fonte: Adaptado de: Elmasri e Navathe (2011).

# Introdução a banco de dados relacionais

- O **Modelo Entidade Relacionamento (MER)** enxerga os dados do mundo real como o conjunto: entidade, atributos e relacionamento.
- Cada entidade, ou um conjunto de entidades, gera uma tabela, seus atributos ou características são representados por colunas desta tabela e cada linha desta tabela representa uma instância dessa entidade.

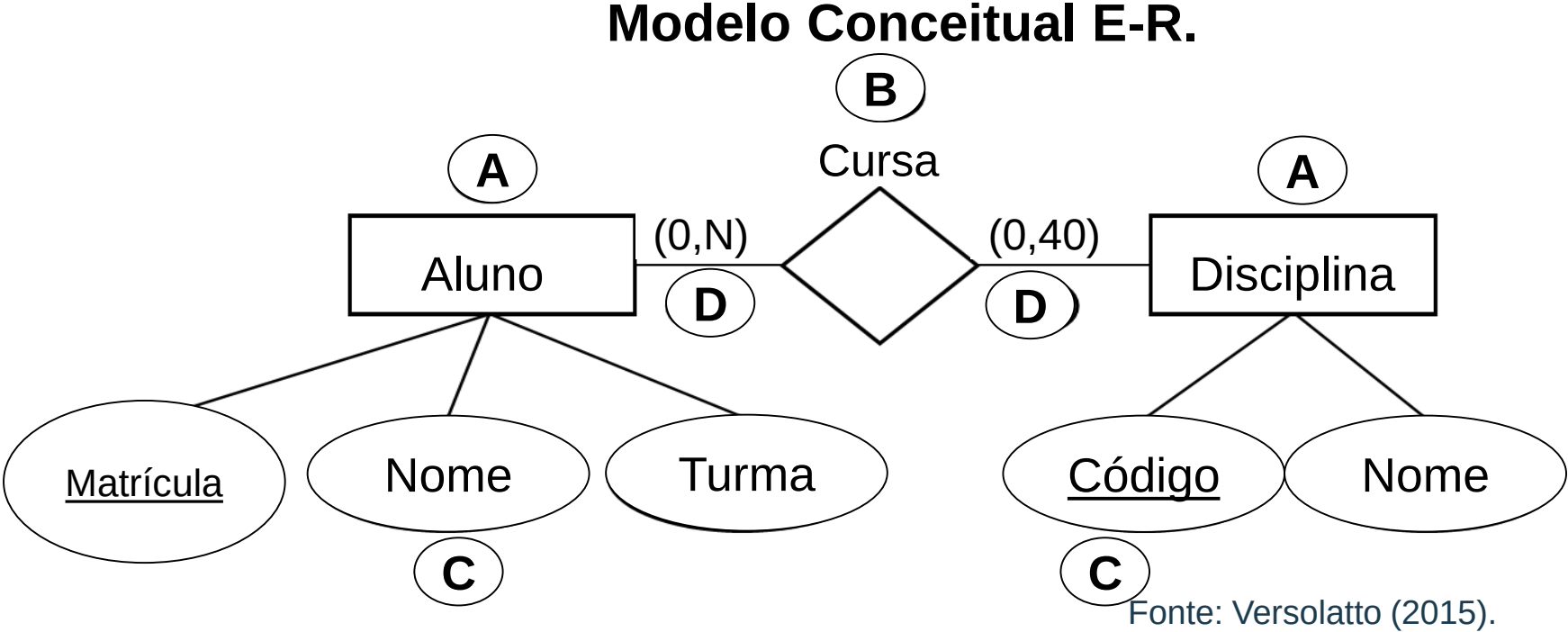
## Modelo Conceitual E-R.



# Banco de dados relacionais – Fase 1 – MER

## Modelo Conceitual E-R

- Cenário de negócio para inscrição de alunos em disciplinas.



Fonte: Versolatto (2015).

| Letra | Descrição                                                                                       |
|-------|-------------------------------------------------------------------------------------------------|
| A     | Entidades: representação de algo do mundo real                                                  |
| B     | Relacionamento: indicação do relacionamento entre duas entidades.                               |
| C     | Atributos: conjunto de características de uma entidade.                                         |
| D     | Multiplicidade: denota a proporção, ou a quantificação, do relacionamento entre duas entidades. |

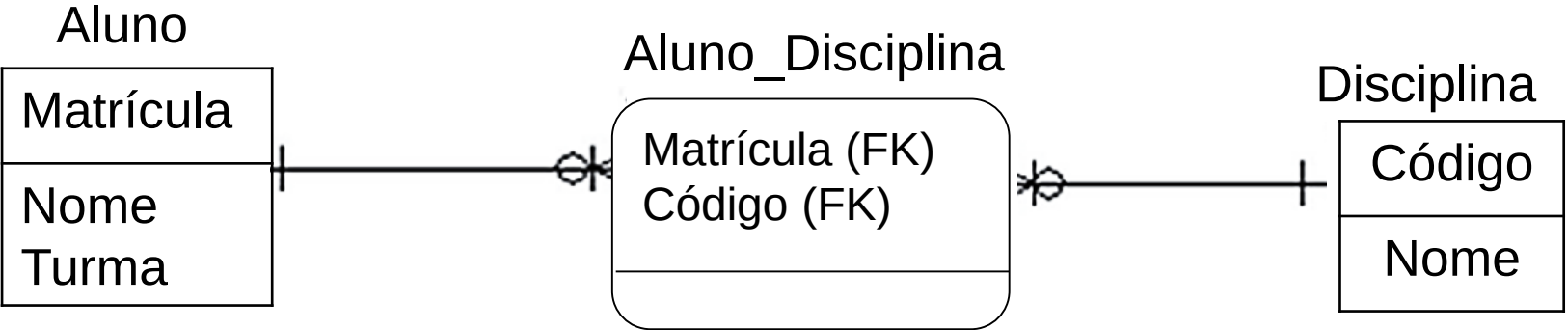
# Banco de dados relacionais – Fase 2 – Dicionário de Dados

- Projeto lógico para cenário de negócio para inscrição de alunos em disciplinas.
- Conversão das entidades em tabelas **E-R do SGBD**.

| MATRÍCULA | NOME            | TURMA |
|-----------|-----------------|-------|
| 12345     | Maria de Maria  | ADS3  |
| 54321     | João da Silva   | ADS3  |
| 667788    | Pedro de Pietro | ADS4  |

| CÓDIGO | NOME                              |
|--------|-----------------------------------|
| BD001  | Banco de dados                    |
| POO1   | Programação Orientada a Objetos 1 |
| WEB01  | Desenvolvimento Web               |

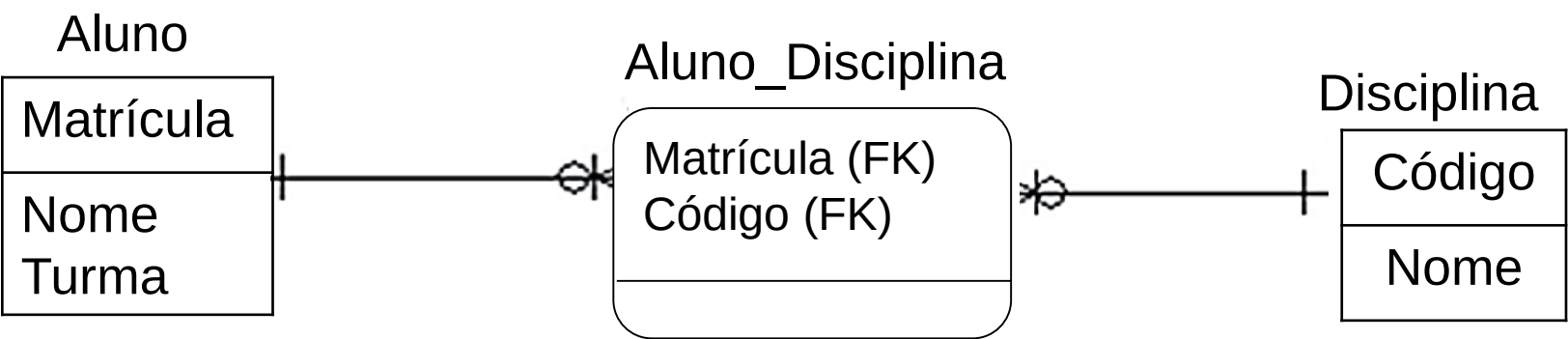
Modelo lógico (Diagrama E-R).



# Banco de dados relacionais – Dicionário de Dados (Cardinalidade)

- Na conversão do dicionário de dados para um diagrama E-R, 1 “Aluno” pode se matricular em nenhuma (0) até uma quantidade (N) “Disciplina”. E 1 “Disciplina” por vez está disponível para (0) ou (N) “Matrícula” com “Código”.

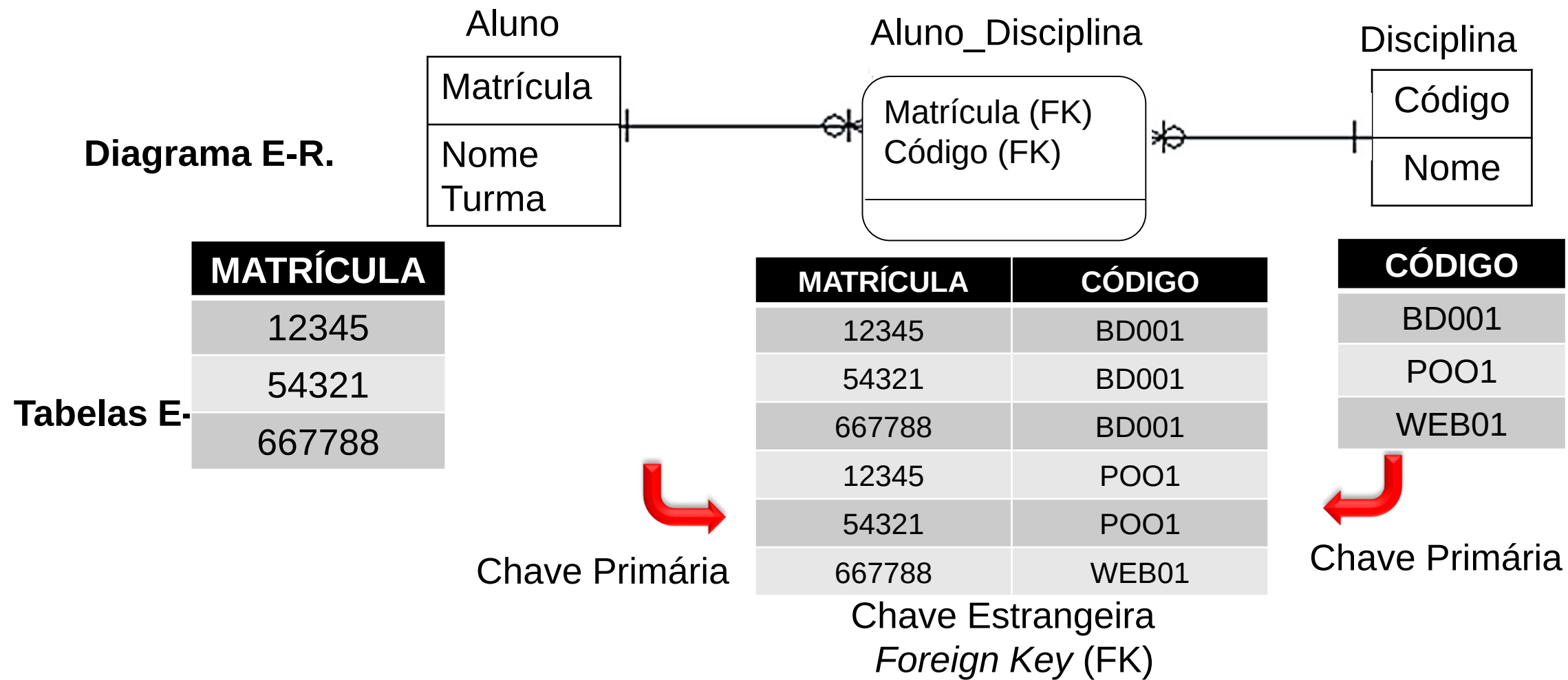
Modelo lógico (Diagrama E-R).



Verifique a cardinalidade:

| Cardinalidade | Representação |
|---------------|---------------|
| (0,1)         | —○—           |
| (1,1)         | —             |
| (0,N)         | —○— /         |
| (1,N)         | — /           |

# Banco de dados relacionais – Chaves primárias e estrangeiras



A partir das duas chaves primárias, uma terceira entidade é gerada. A esta entidade é dado o nome de chave estrangeira.

# Interatividade

Relacionar as entidades “Fábrica” e “Cliente” para gerar uma nova entidade de nome “Entrega”. Assinale a alternativa correspondente à especificação desta função.

- a) Entidade Entrega com atributos estrangeiros Fábrica (FK) e Cliente (FK).
- b) Entidade Entrega com atributos primários Fábrica (FK) e Cliente (FK).
- c) Entidade Entrega com atributos estrangeiros Produto (FK): Quantidade (FK), Valor (FK), data\_Compr (FK); Nome\_Cliente (FK), endereco\_Cliente (FK) e data\_Entreg (FK).
- d) Entidade Entrega com atributos primários Produto (FK): Quantidade (FK), Valor (FK), data\_Compr (FK), Nome\_Cliente (FK), endereco\_Cliente (FK) e data\_Entreg (FK).
- e) Entidade Entrega com atributos primários Produto (FK), Cliente (FK) e data\_Entreg (FK).

| Fábrica | Produto           | Quantidade              | Valor       | data_Compr  |
|---------|-------------------|-------------------------|-------------|-------------|
|         | TV Vision         | 20                      | R\$ 3500,00 | 01/06/2020  |
|         |                   |                         |             |             |
| Cliente | Nome_Cliente      | endereco_Cliente        |             | data_Entreg |
|         | Mônica Palistener | Rua Cocha Bom, 123 – SP |             | 08/06/2020  |

## Resposta

Relacionar as entidades “Fábrica” e “Cliente” para gerar uma nova entidade de nome “Entrega”. Assinale a alternativa correspondente a especificação desta função.

- a) Entidade Entrega com atributos estrangeiros Fábrica (FK) e Cliente (FK).
- b) Entidade Entrega com atributos primários Fábrica (FK) e Cliente (FK).
- c) Entidade Entrega com atributos estrangeiros Produto (FK): Quantidade (FK), Valor (FK), data\_Compr (FK); Nome\_Cliente (FK), endereco\_Cliente (FK) e data\_Entreg (FK).
- d) Entidade Entrega com atributos primários Produto (FK): Quantidade (FK), Valor (FK), data\_Compr (FK), Nome\_Cliente (FK), endereco\_Cliente (FK) e data\_Entreg (FK).
- e) Entidade Entrega com atributos primários Produto (FK), Cliente (FK) e data\_Entreg (FK).

| Fábrica | Produto           | Quantidade              | Valor       | data_Compr  |
|---------|-------------------|-------------------------|-------------|-------------|
|         | TV Vision         | 20                      | R\$ 3500,00 | 01/06/2020  |
|         |                   |                         |             |             |
| Cliente | Nome_Cliente      | endereco_Cliente        |             | data_Entreg |
|         | Mônica Palistener | Rua Cocha Bom, 123 – SP |             | 08/06/2020  |

# Banco de dados relacionais – Modelo físico implementado em SQL

Chave Primária

| MATRÍCULA |
|-----------|
| 12345     |
| 54321     |
| 667788    |

Chave Primária

| CÓDIGO |
|--------|
| BD001  |
| POO1   |
| WEB01  |

Fonte: Adaptado de: Versolatto (2015).

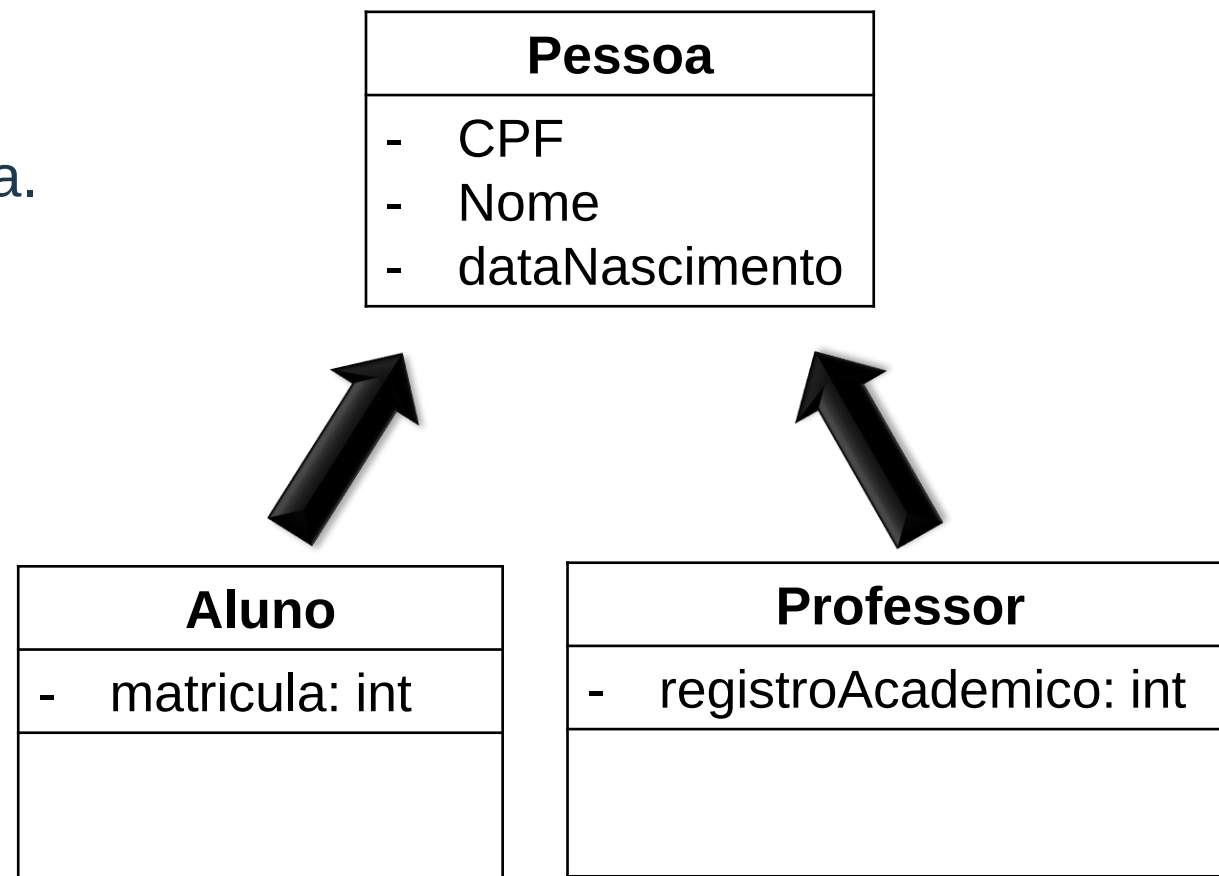
Chave Estrangeira

| MATRÍCULA | CÓDIGO |
|-----------|--------|
| 12345     | BD001  |
| 54321     | BD001  |
| 667788    | BD001  |
| 12345     | POO1   |
| 54321     | POO1   |
| 667788    | WEB01  |

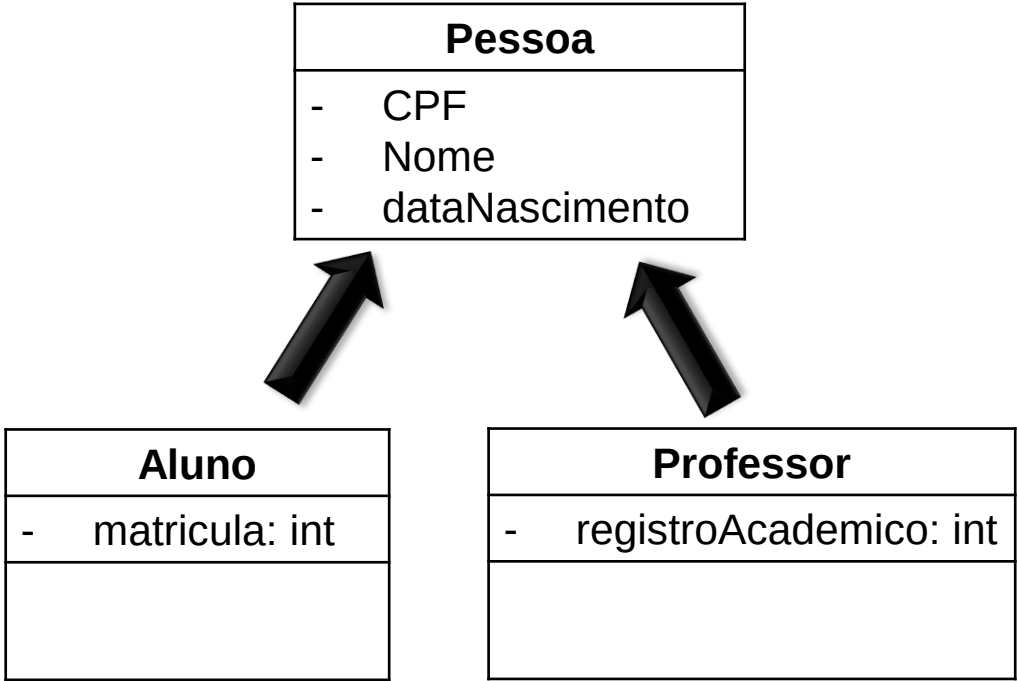
```
CREATE TABLE ALUNO
(
  MATRICULA INT NOT NULL,
  NOME VARCHAR (20) NOT NULL,
  TURMA VARCHAR (10) NOT NULL,
  PRIMARY KEY (MATRICULA)
)
CREATE TABLE DISCIPLINA
(
  CODIGO VARCHAR (20) NOT NULL,
  NOME VARCHAR (100) NOT NULL,
  PRIMARY KEY (CODIGO)
)
CREATE TABLE ALUNO_DISCIPLINA
(
  MATRICULA_ALUNO,
  CODIGO_DISCIPLINA,
  FOREIGN KEY (MATRICULA_ALUNO)
    REFERENCES ALUNO (MATRICULA),
  FOREIGN KEY (CODIGO_DISCIPLINA)
    REFERENCES DISCIPLINA (CODIGO)
)
```

# Banco de dados relacionais *versus* orientação a objetos

- A diferença básica entre o BD Relacional (BD E-R) e o BD (BD OO) Orientado a Objetos é o armazenamento dos métodos que agem sobre uma determinada classe.
- As semelhanças são várias: as entidades possuem atributos; a entidade possui uma identidade; e as entidades se relacionam.
- Algumas características do BD OO não se aplicam ao BD E-R, como é o caso da herança. No BD E-R não existe o recurso de herança.



# BD Relacional – Como mapear a herança?



Solução 1) Criar uma única entidade contendo todas as informações.

|     |      |                |           |                   |
|-----|------|----------------|-----------|-------------------|
| CPF | Nome | dataNascimento | matricula | registroAcademico |
|-----|------|----------------|-----------|-------------------|

Solução 2) Criar duas entidades: uma para Aluno e uma para Professor.

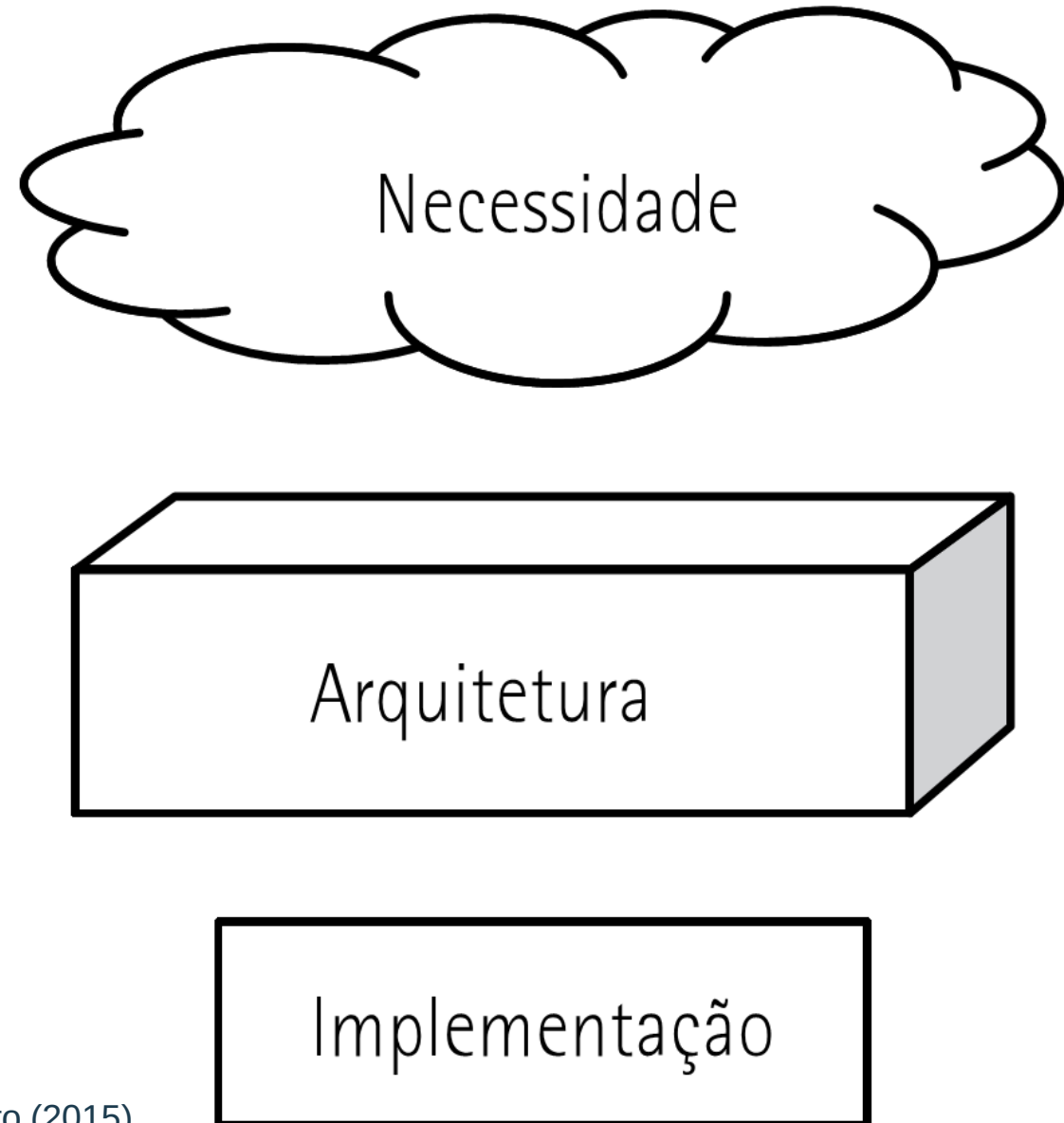
|     |      |                |                   |
|-----|------|----------------|-------------------|
| CPF | Nome | dataNascimento | matricula         |
| CPF | Nome | dataNascimento | registroAcademico |

Solução 3) Criar três entidades: uma para Pessoa, uma para Aluno e uma para Professor.

|     |                   |                |
|-----|-------------------|----------------|
| CPF | Nome              | dataNascimento |
| CPF | matricula         |                |
| CPF | registroAcademico |                |

# Projeto arquitetural

- A figura mostra onde a arquitetura se encontra dentro do contexto do ciclo de vida do projeto de *software*.
- Arquitetura do *software* é o nome dado à disciplina fundamental da fase de projeto arquitetural.



# Projeto arquitetural – O que é arquitetura de *software*?

Definições clássicas de arquitetura de *software* são:

- Arquitetura de *software* é uma descrição de como um sistema de *software* é organizado (SOMMERVILLE, 2010).
- Arquitetura é a organização fundamental de um sistema incorporada em seus componentes, seus relacionamentos com o ambiente, e nos princípios que conduzem seu projeto, construção e evolução (ISO, 2011b).
- Arquitetura de *software* pode ser definida como uma representação, em alto nível de abstração, dos componentes de um sistema de *software*, suas propriedades externamente visíveis, e como estes interagem com o objetivo de resolver as necessidades, ou problema de um cenário de negócio (BASS; CLEMENTS; KAZMAN, 2010).

Saiba mais:

A norma ISO/IEC/IEEE 42010 (2011b) tem como objetivo padronizar a prática da arquitetura definindo termo, apresentando uma base conceitual para expressar, comunicar e rever arquiteturas e especificações de requisitos que se aplicam às descrições de arquitetura [...]. Esta norma substitui a norma ISO 1471 de 2000.

# Projeto arquitetural – A importância da arquitetura de *software*

- A arquitetura de *software* produz modelos e organiza a estrutura de um *software* que serve de guia para a construção do *software*.
- Segundo o SEI (2015a), a arquitetura de *software* serve como modelo tanto para o projeto quanto para a construção, facilitando a divisão de tarefas pela equipe de projeto e de construção do *software*.
- A arquitetura de *software* é o principal meio para o atingimento da qualidade do sistema, como: desempenho, manutenibilidade e segurança. Nenhum índice de qualidade pode ser alcançado sem uma visão arquitetural unificada (SEI, 2015a).

## Saiba mais:

A arquitetura de *software* está intimamente ligada aos atributos de qualidade, ou requisitos não funcionais, que podem ser vistos na norma ISO 25010 (ISO, 2011a).

Fonte: Bass, Clements e Kazman (2010); o SEI (2015a).

# Projeto arquitetural – A arquitetura de *software* e seus benefícios

Bass, Clements e Kazman (2010) pontuam a importância da arquitetura de *software*:

- Facilita a comunicação entre os envolvidos no projeto.
- Representa, em escala menor e compreensível, a forma pela qual os componentes de um sistema são estruturados e interagem.
- Modela e evidencia decisões de projeto como fator fundamental para o sucesso do sistema.

Benefícios de uma arquitetura bem definida para um sistema:

- Facilita a evolução do *software*.
- Facilita o reúso de componentes.
- Promove redução do *lead time* entre as fases de análise e construção e, por isso, diminui o retrabalho na construção.
- Auxilia a gestão do projeto quanto à estimativa de tempo, de custo e à da complexidade do projeto.
- Promove mitigação do risco.

# Projeto arquitetural – Processo de arquitetura de *software*

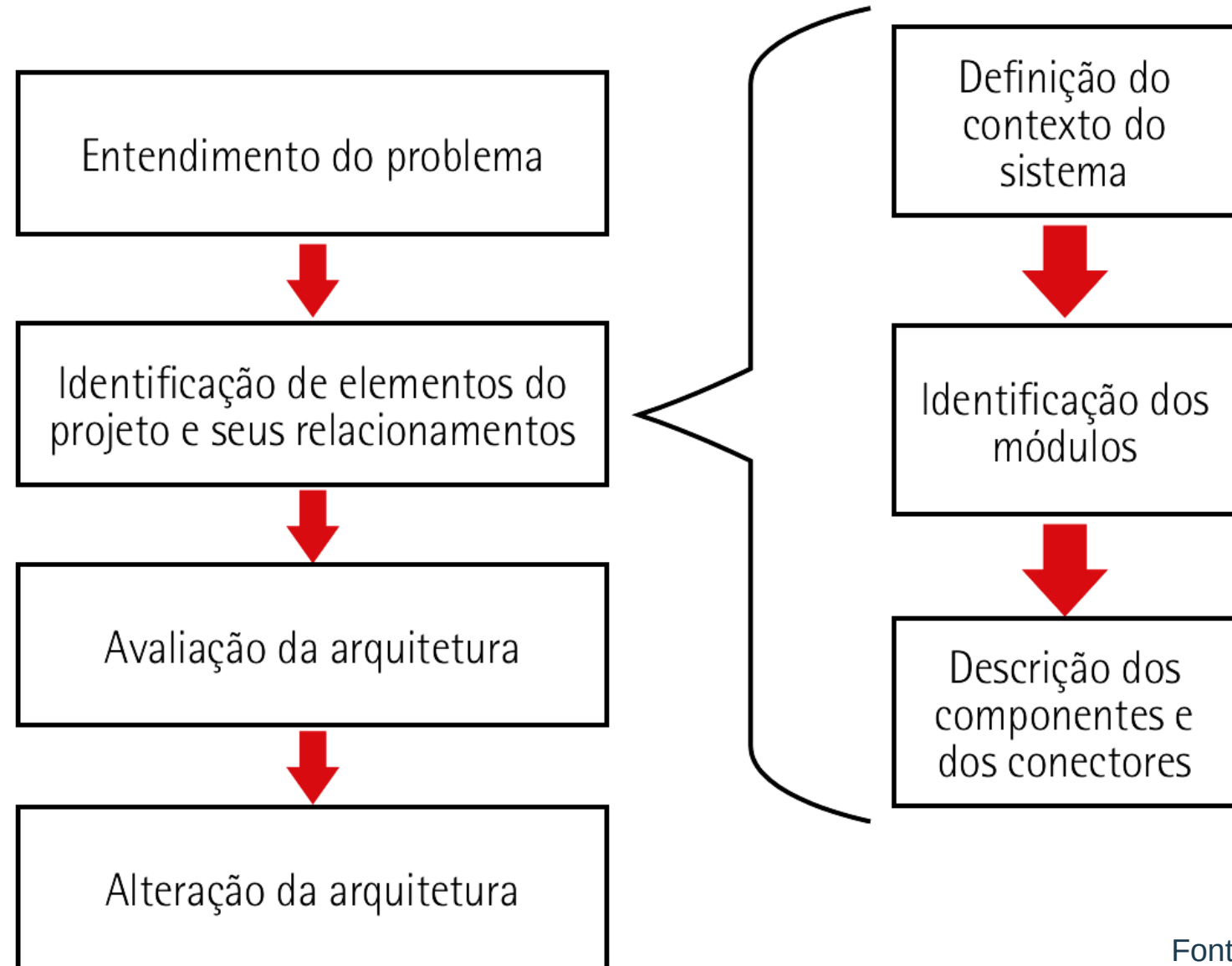
Segundo Albin (2003), a arquitetura de *software* pode ser dividida em cinco fases, e deve ser iniciada antes mesmo da fase de projetos do ciclo de vida da engenharia de *software*:

| Fases do ciclo de vida da engenharia de <i>software</i> |   | Fases da arquitetura       |
|---------------------------------------------------------|---|----------------------------|
| Análise de requisitos                                   | ➡ | Pré-projeto                |
| Projeto                                                 | ➡ | Análise de domínio         |
|                                                         |   | Projeto esquemático        |
| Implementação e Testes                                  | ➡ | Desenvolvimento de projeto |
|                                                         |   | Construção                 |
| Implantação                                             | ➡ | Entrega                    |

Fonte: Albin (2003).

# Projeto arquitetural – Atividades do processo de arquitetura de *software*

- Sequência de atividades do processo de definição de arquitetura sugerido por Albin (2003).



# Projeto arquitetural – Aspectos humanos do processo

- Segundo o OpenUP ([s.d.]a), os principais interessados no processo de definição de arquitetura de um sistema de *software*: analistas de requisitos, desenvolvedores, gerentes de projetos e arquitetos de *software*.

Além de converter os requisitos do cliente em projeto de *software*, as principais responsabilidades do arquiteto do *software* são:

- Promover um estudo da viabilidade das soluções técnicas adotadas.
- Avaliar as tendências tecnológicas, em vista das soluções adotadas no projeto, sob o ponto de vista de competitividade do produto a ser desenvolvido.
- Buscar o aumento do tempo de vida útil do *software*.

Saiba mais:

SENE, Rafael Peria de. OpenUP: uma visão geral.

Disponível em

<https://www.tiespecialistas.com.br/openup-uma-visao-geral/>,  
23/09/2010. Consultado em 23/06/2020.

# Interatividade

Analise cada afirmativa como Verdadeira (V) ou Falsa (F) e assinale a alternativa correta:

- I. ( ) Arquitetura de *software* é uma descrição de como um sistema de *software* é organizado.
- II. ( ) O arquiteto do *software* deve identificar os elementos do projeto e seus relacionamentos.
- III. ( ) O arquiteto do *software* deve conhecer os aspectos culturais de seu cliente.

- a) F, F, V.
- b) F, V, F.
- c) V, F, V.
- d) V, V, F.
- e) V, V, V.

# Resposta

Analise cada afirmativa como Verdadeira (V) ou Falsa (F) e assinale a alternativa correta:

- I. (V) Arquitetura de *software* é uma descrição de como um sistema de *software* é organizado.
- II. (V) O arquiteto do *software* deve identificar os elementos do projeto e seus relacionamentos.
- III. (V) O arquiteto do *software* deve conhecer os aspectos culturais de seu cliente.

- a) F, F, V.
- b) F, V, F.
- c) V, F, V.
- d) V, V, F.
- e) V, V, V.

Saiba mais:

DS - OpenUP. Função: Arquiteto de *Software*.

Disponível em: [https://www.trt9.jus.br/pds/pdstrt9/roles/architect\\_E7A12309.html](https://www.trt9.jus.br/pds/pdstrt9/roles/architect_E7A12309.html). Consultado em 23/06/2020.

# Visões da arquitetura de *software*

A arquitetura do *software* define os componentes que formarão o sistema. Esses componentes podem ser divididos em dois grupos:

1. A visão estática da arquitetura promove a visão da organização e relações dos componentes do *software* com elementos de dados (banco de dados, arquivos-texto etc.), com o *hardware* e outros ambientes operacionais.
2. A visão dinâmica da arquitetura promove a visão comportamental do sistema e de seus componentes. São definidos como os componentes reagem a eventos internos e externos e a forma como eles se comunicam (ou trocam mensagens).

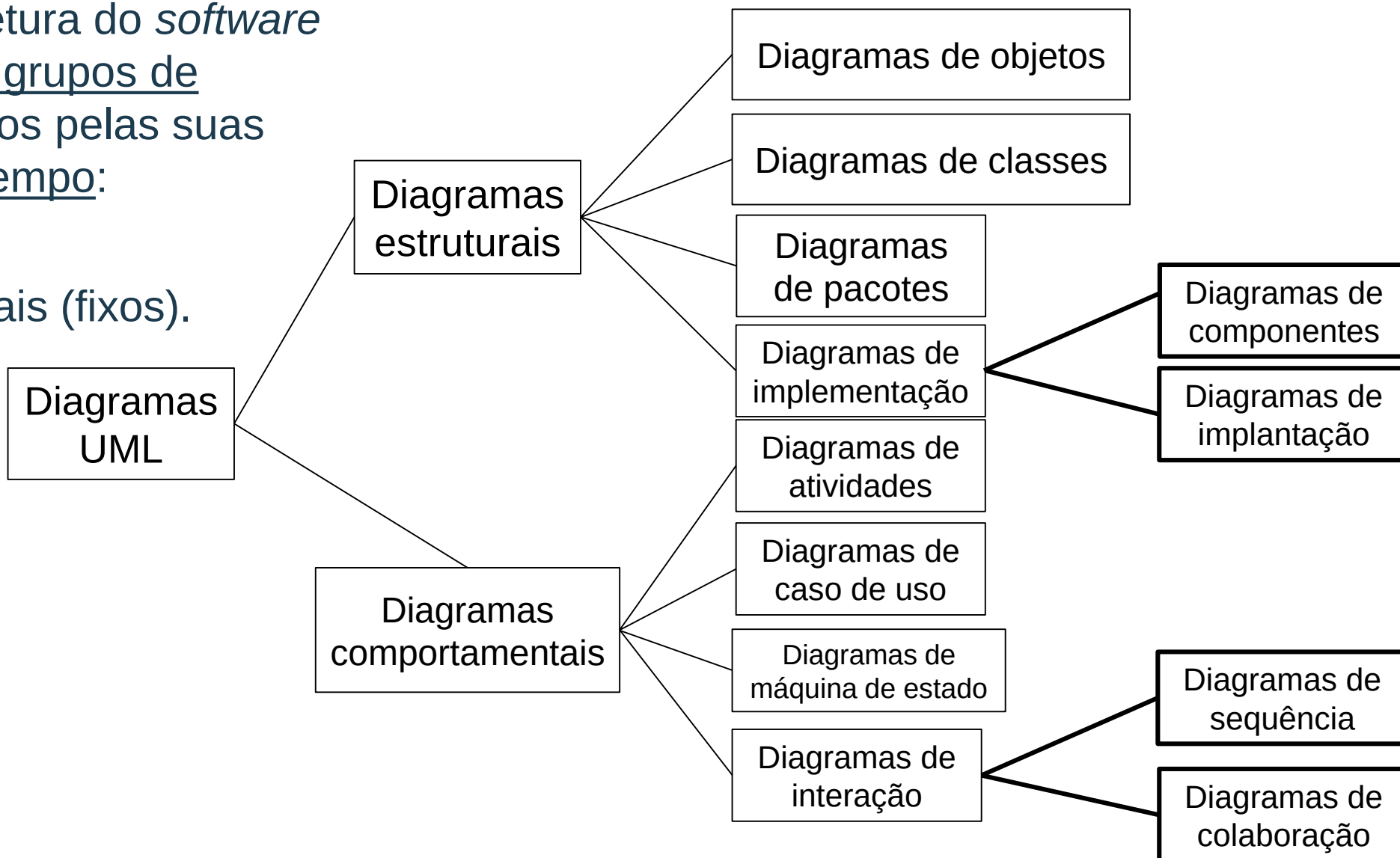
Saiba mais:

SILVA, Tomas Anderson Souza. *Modelos UML estáticos vs. dinâmicos*. Disponível em <http://tassinfo.com.br/orientacao-a-objeto/11-modelos-uml-estaticos-vs-dinamicos/>, 19/04/2020. Consultado em 23/06/2020.

# Arquitetura do *software* – Revendo UML

Observe que na arquitetura do *software* com UML existem dois grupos de diagramas, determinados pelas suas variações na linha do tempo:

1. Diagramas estruturais (fixos).
2. Diagramas comportamentais (dinâmicos).



# Arquitetura do *software* – Visão estática

- Projetar uma estrutura de *software* significa escolher as alternativas de soluções adequadas e inerentes ao domínio do problema.
- **A viabilidade do sistema** mede o domínio do problema com base nos aspectos culturais, infraestrutura de TI do cliente, expectativa de competitividade do cliente, custo e prazo em relação ao *software* a ser desenvolvido.
- **A visão estática do sistema** é caracterizada pela integração dos elementos que compõem os sistemas de informação: *Software*, *Hardware*, Pessoas, Banco de dados e Rede de computadores.



**Comece a  
interpretar a  
infraestrutura de  
TI.**

Saiba mais: ◦

IBM Knowledge Center. Diagramas de Implementação.

Disponível em

[https://www.ibm.com/support/knowledgecenter/pt-](https://www.ibm.com/support/knowledgecenter/pt-br/SS4JE2_7.5.5/com.ibm.xtools.modeler.doc/topics/cdepd.html)

[br/SS4JE2\\_7.5.5/com.ibm.xtools.modeler.doc/topics/cdepd.html](https://www.ibm.com/support/knowledgecenter/pt-br/SS4JE2_7.5.5/com.ibm.xtools.modeler.doc/topics/cdepd.html).

Consultado em 24/06/2020.

# Arquitetura do *software* – Visão estática – Padrões na arquitetura

- Gamma *et al.* (1995), considerado como referência em padrões na arquitetura de *software*, define padrão como uma solução reutilizável descrita com base em três partes: um contexto, um problema e uma solução.
- **Contexto:** estende o problema a ser solucionado, apresentando situações de ocorrência desses problemas.
- **Problema:** determinado por um sistema de forças, em que estas forças estabelecem os aspectos do problema que devem ser considerados.
- **Solução:** mostra como resolver o problema recorrente e como balancear as forças associadas a ele.

Saiba mais:

GAMMA, Erich; HELM, Richard; JOHNSON, Ralph; VLISSIDES, John. *Padrões de Projetos: Soluções Reutilizáveis de Software Orientados a Objetos*. Porto Alegre: Bookman, 2007. (Versão Kindle Edition).

# Arquitetura do *software* – Visão estática – Estilo arquitetural

Os principais estilos arquiteturais, extraídos de Buschmann *et al.* (1996), são classificados em quatro categorias.

| Categoria                                               | Estilo                                        |
|---------------------------------------------------------|-----------------------------------------------|
| <i>From mud to structure</i><br>(Da “lama” à estrutura) | Camadas ou <i>Layers</i>                      |
|                                                         | <i>Pipes and Filters</i>                      |
|                                                         | <i>Blackboard</i>                             |
| Sistemas distribuídos                                   | <i>Broker</i>                                 |
|                                                         | <i>Microkernel</i>                            |
|                                                         | <i>Pipes and Filters</i>                      |
| Sistemas interativos                                    | <i>Model-View-Controller (MVC)</i>            |
|                                                         | <i>Presentation-Abstraction-Control (PAC)</i> |
| Sistemas adaptáveis                                     | <i>Reflection</i>                             |
|                                                         | <i>Microkernel</i>                            |

Fonte: Buschmann *et al.* (1996).

# Arquitetura do *software* – Visão estática – Estruturação de sistemas I

- Caso: Ciclo de vida do desenvolvimento de um sistema para cadastro de funcionários.
- Ambiente operacional: Sistema *web*.

Ambiente de desenvolvimento:

- Requisitos do usuário → Regras de negócio → Página *web* → Banco de dados.
- Acréscimo de novas funções → Reutilização de código para implementação. (Reusabilidade)
- **Problema ou necessidade:** Mudanças nas regras de negócio, após período de uso.
- **Análise:** As alterações não parecem complexas e na verdade elas realmente não são, todavia você esbarra, agora, em alguns problemas!



Fonte: Clipart.

# Arquitetura do *software* – Visão estática – Estruturação de sistemas II

## Problemas decorrentes de mudanças nos requisitos:

- Rastreamento do código, evolução do sistema, alteração do código e testes.
- Acoplamento de fatores sem relação direta com a mudança, como o banco de dados com a página *web*. Mudanças isoladas em componentes do sistema podem afetar todo o sistema.
- Aumento no custo da manutenção.



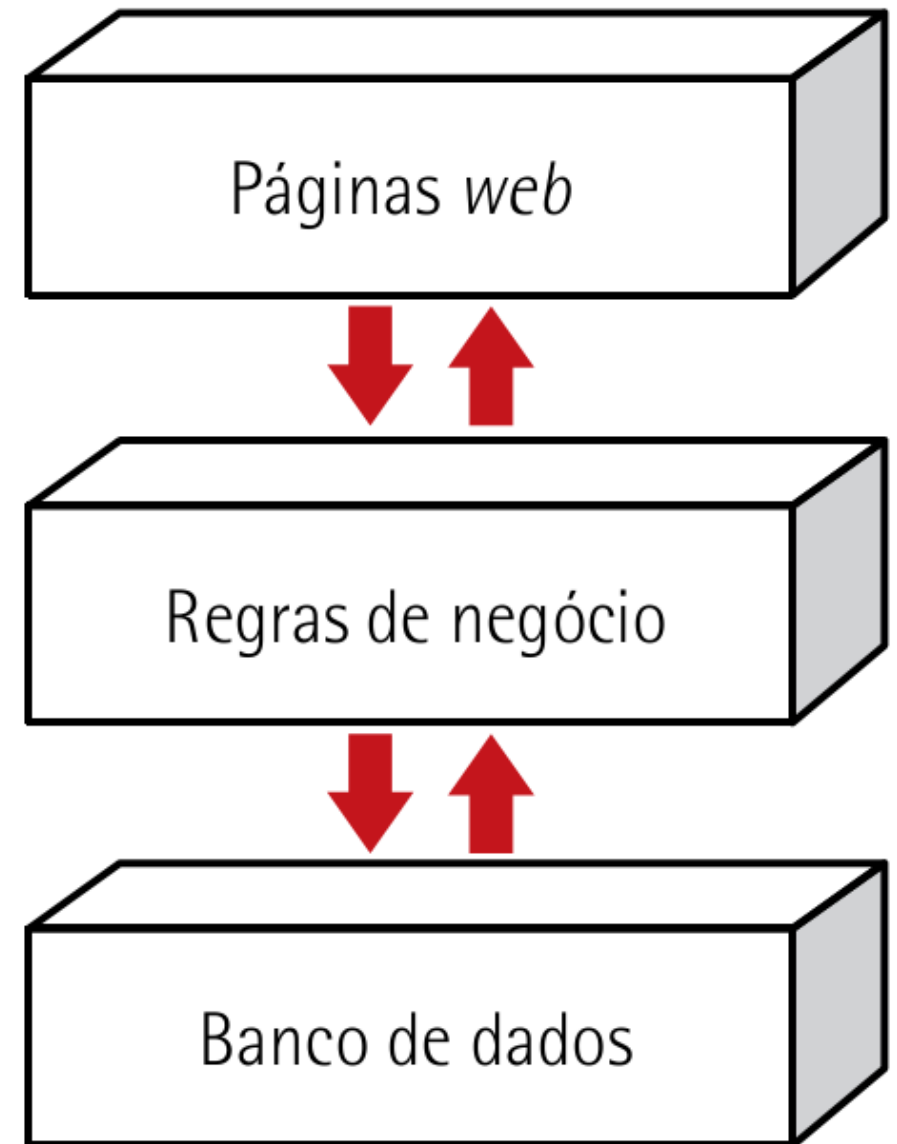
Avaliação de qualidade

| Métrica              | Definição                                                   |
|----------------------|-------------------------------------------------------------|
| Acoplamento e Coesão | Grau de acoplamento que pode afetar atributos externos.     |
| Manutenibilidade     | Capacidade do produto de <i>software</i> de ser modificado. |
| Reusabilidade        | Flexibilidade de uso do código.                             |

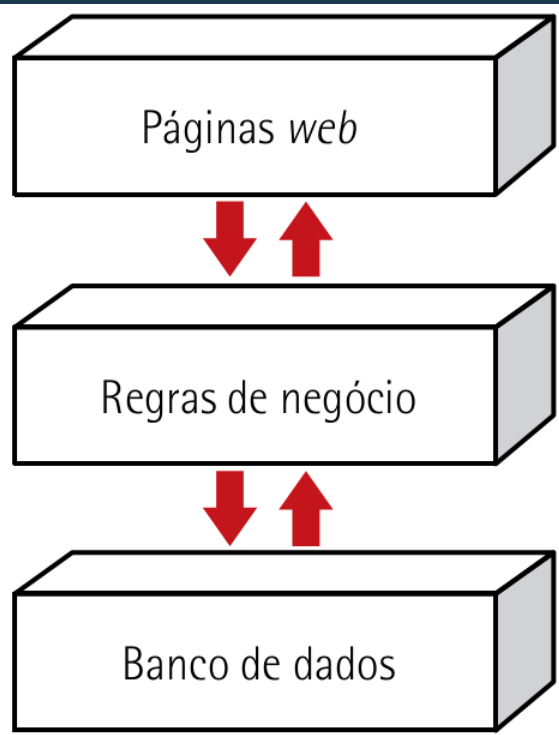
# Arquitetura do *software* – Visão estática – Arquitetura em camadas I

- Para visualizar o ambiente operacional do cliente, é necessário dividir o sistema em camadas, em que cada “caixa” corresponde a uma camada.
- Cada “caixa” possui uma função específica no sistema e pode representar os módulos do sistema.

A arquitetura de camadas possui a estrutura final representada na figura:



# Arquitetura do software – Visão estática – Avaliação da qualidade



| Avaliação da qualidade       |                                                                                                                                        |
|------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Resultados                   | Definição                                                                                                                              |
| Divisão de responsabilidades | Cada “caixa” tem sua responsabilidade definida.                                                                                        |
| Acoplamento                  | As “caixas” teriam apenas as dependências necessárias. Páginas <i>web</i> , por exemplo, não precisam compor código do banco de dados. |
| Manutenibilidade             | Os componentes são específicos para cada “caixa”. As mudanças são isoladas, independentes de outros módulos.                           |
| Reúso                        | Um componente específico pode ser reusado, sem a necessidade de compor este código em outro módulo.                                    |

# Interatividade

A visão estática da arquitetura do *software* permite apresentar a arquitetura do sistema em camadas. Com esta visão, é possível dar manutenção em cada componente isoladamente, **PORQUE** cada camada possui vários componentes como referência para que o código possa ser copiado e colado em outras camadas. Assinale a alternativa correta de análise do texto:

- a) Argumento inválido e justificativa inválida.
- b) Argumento inválido e justificativa válida. O componente representa conjunto de camadas.
- c) Argumento válido e justificativa inválida. O código é reusado e não copiado e colado.
- d) Argumento válido e justificativa válida. A justificativa completa o argumento.
- e) Argumento válido e justificativa válida. A justificativa não responde ao argumento.

## Resposta

A visão estática da arquitetura do *software* permite apresentar a arquitetura do sistema em camadas. Com esta visão, é possível dar manutenção em cada componente isoladamente, **PORQUE** cada camada possui vários componentes como referência para que o código possa ser copiado e colado em outras camadas. Assinale a alternativa correta de análise do texto:

c) Argumento válido e justificativa inválida. O código é reusado e não copiado e colado.

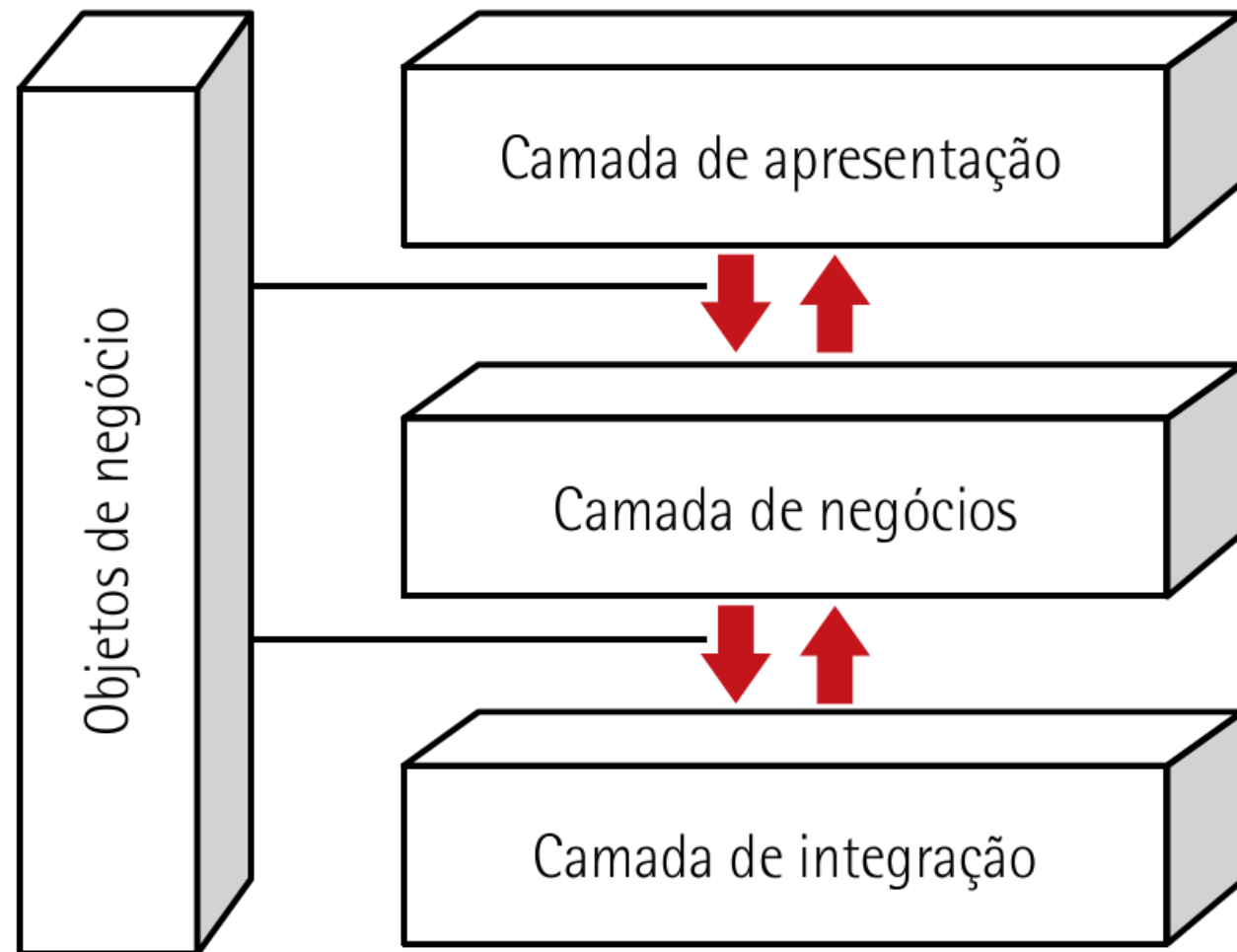
### Característica da arquitetura em camadas.

|              |                                                                                                     |
|--------------|-----------------------------------------------------------------------------------------------------|
| <b>Reúso</b> | Um componente específico pode ser reusado, sem a necessidade de compor este código em outro módulo. |
|--------------|-----------------------------------------------------------------------------------------------------|

# Arquitetura do *software* – Visão estática – Arquitetura três camadas

- A **arquitetura três camadas** é um padrão estabelecido na construção de sistemas.

| Camada                  | Definição                                                                              |
|-------------------------|----------------------------------------------------------------------------------------|
| Apresentação            | Classes responsáveis pela interação com o usuário.                                     |
| Negócios (ou Aplicação) | Classes responsáveis pela execução das regras de negócio.                              |
| Integração              | Classes responsáveis pela integração das tecnologias da informação. Por exemplo: SGBD. |



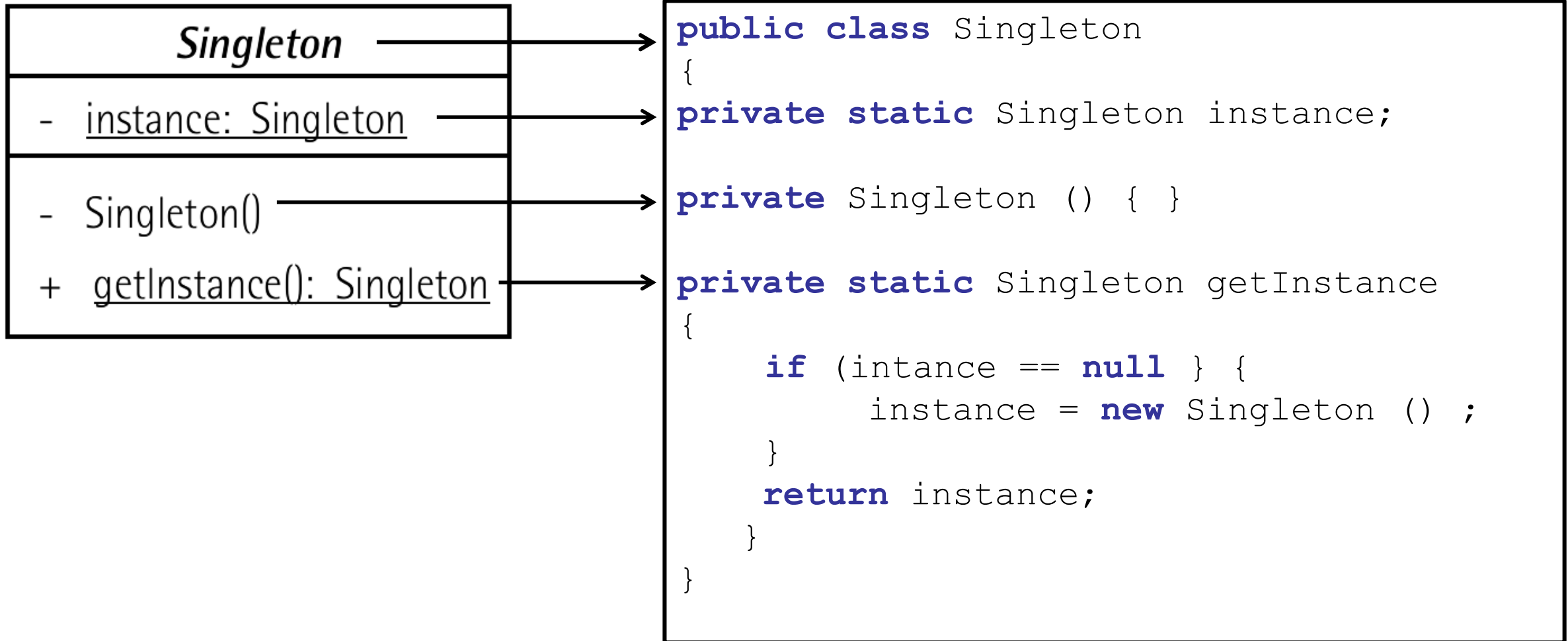
# Introdução a padrões de projeto (*design patterns*)

- Gamma *et al.* (1995) propõem um catálogo com 23 padrões de projetos.
- São três categorias de padrões: para **Criação**, **Estruturais** e **Comportamentais**.
- Tabela: Padrões de projeto para a categoria “Padrões para criação”.

| Padrões para criação    |                                                            |
|-------------------------|------------------------------------------------------------|
| Padrão de Projeto       | Descrição                                                  |
| <i>Abstract factory</i> | Interface para criação de famílias de objetos.             |
| <i>Builder</i>          | Permite criar representações independentes do objeto.      |
| <i>Factory method</i>   | Interface para criação de objeto.                          |
| <i>Prototype</i>        | Cria objetos a partir da instância de um protótipo.        |
| <i>Singleton</i>        | Garante uma única instância da classe dentro da aplicação. |

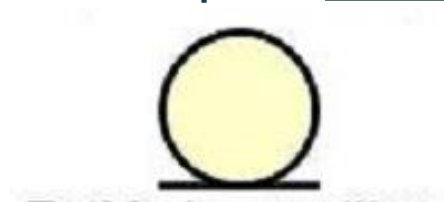
# Introdução a padrões de projeto (*design patterns*) – Aplicação

- O padrão *Singleton*, por exemplo, tem por objetivo utilizar uma única instância de uma classe em todo o sistema. A figura representa a sintaxe de aplicação para o padrão *Singleton*.



# Visão dinâmica

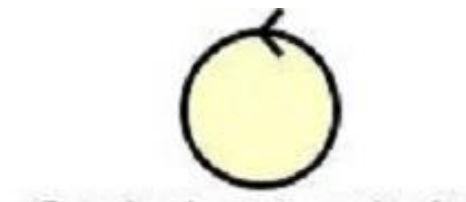
- O objetivo principal da visão dinâmica da arquitetura é representar a realização dos casos de uso, que se dá pela troca de mensagens entre os objetos no decorrer do tempo.
- Inicialmente deve-se determinar as responsabilidades dos objetos. A responsabilidade de cada objeto é dada pela classificação das classes, como:



Entidade<<entity>>



Fronteira<<boundary>>

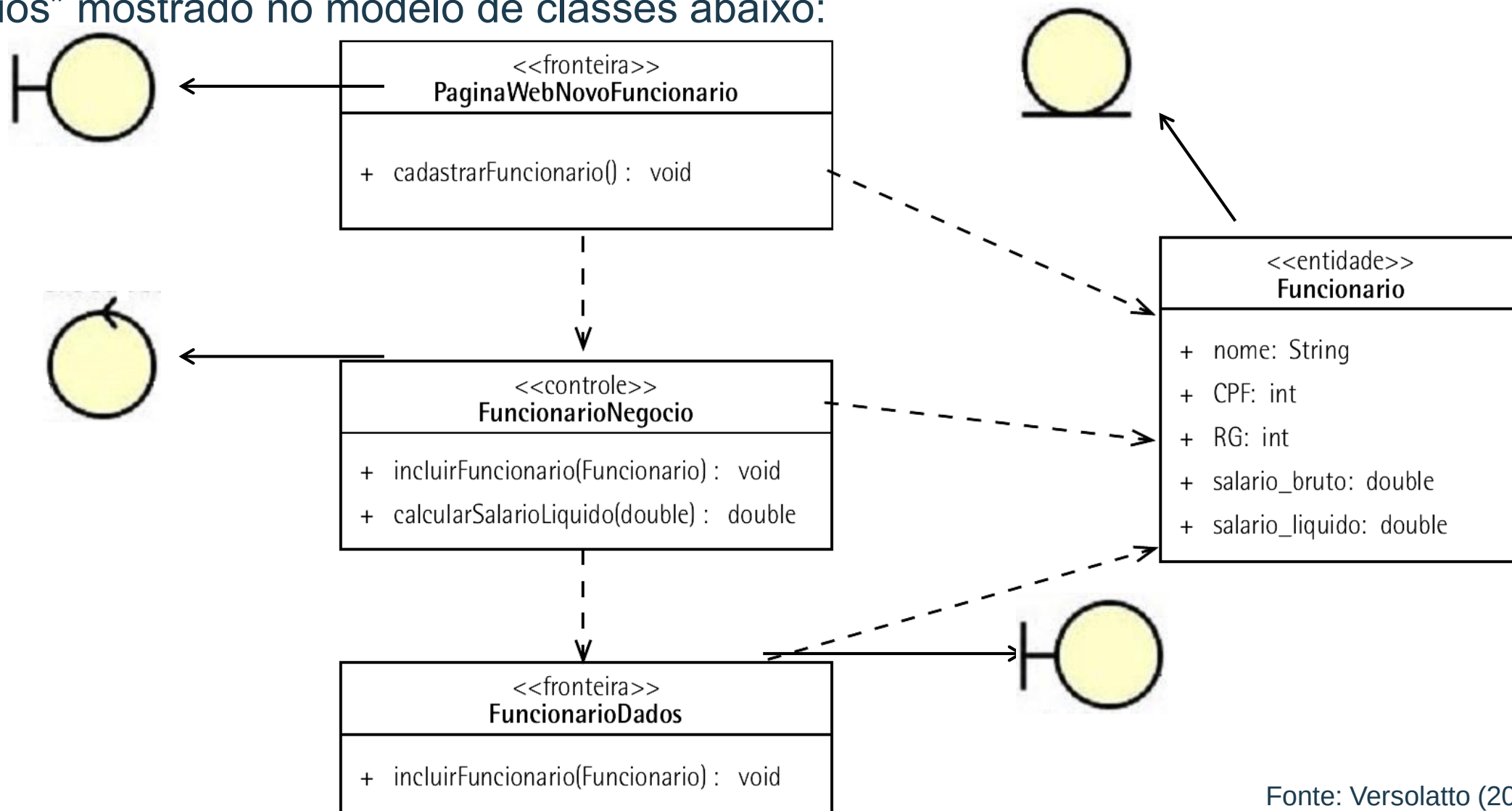


Controle<<control>>

| Classificação             | Descrição                                                                                 |
|---------------------------|-------------------------------------------------------------------------------------------|
| Entidade<br><<entity>>    | Classe passiva que contém informações. Participa das realizações de casos de uso.         |
| Fronteira<br><<boundray>> | Estes objetos fazem a interface de comunicação de atores com objetos internos do sistema. |
| Controle<br><<control>>   | A classe está na camada da aplicação (regras de negócio).                                 |

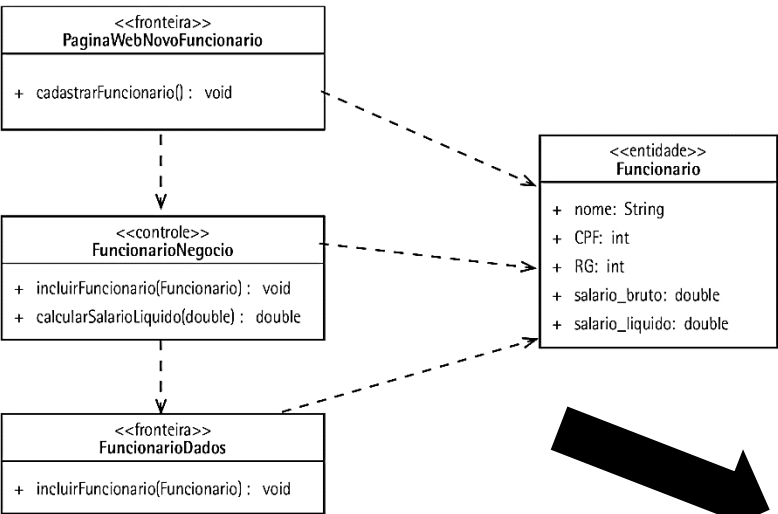
# Visão dinâmica – Definindo responsabilidades

Para exemplificar a definição de responsabilidades das classes, será usada a função “Cadastro de funcionários” mostrado no modelo de classes abaixo:

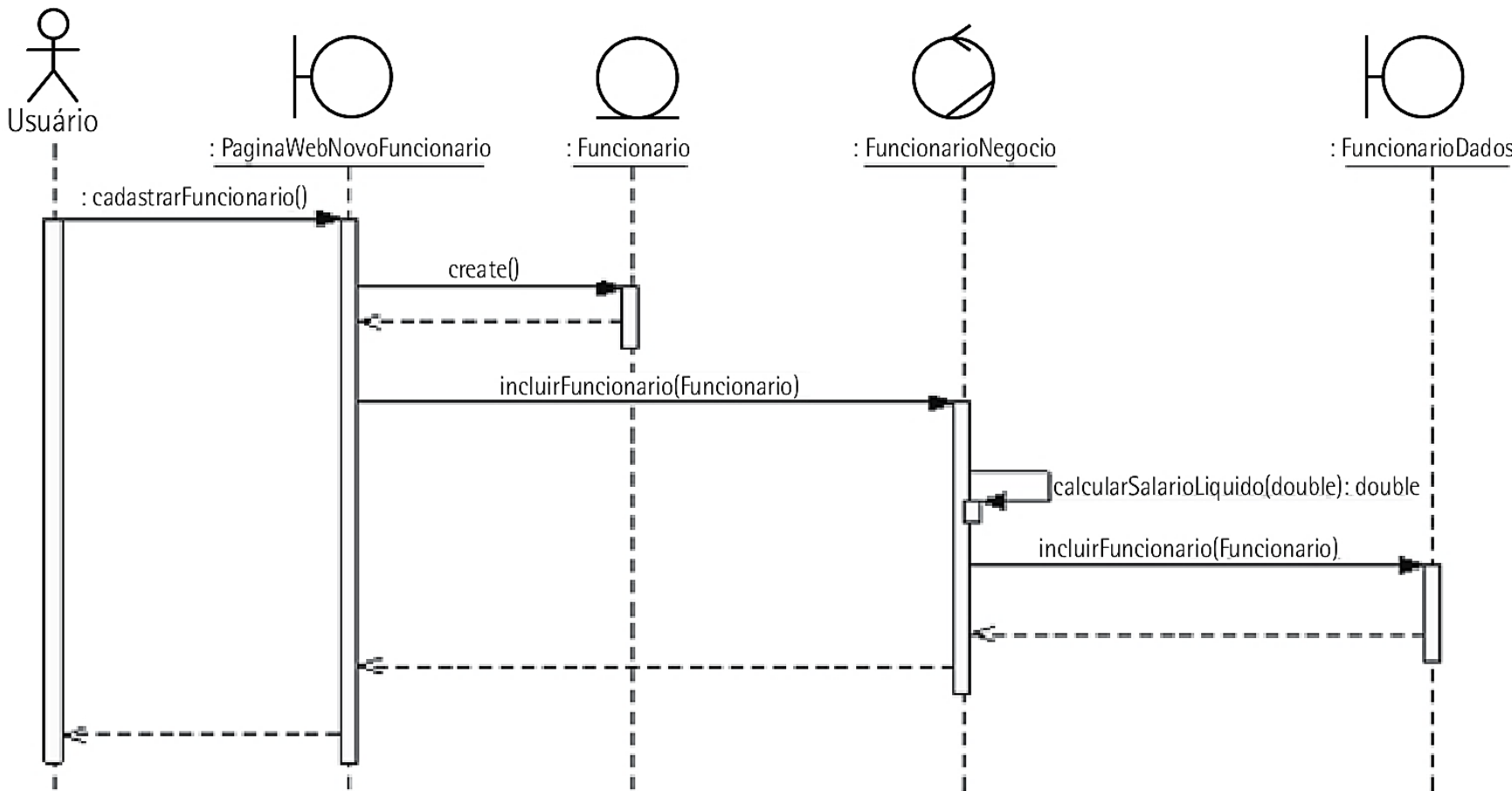


# Visão dinâmica – Refinando o diagrama de sequência

A figura da esquerda apresenta o modelo de classes que dá origem ao modelo de sequência na figura abaixo.



Fonte: Versolatto (2015).



Fonte: Versolatto (2015).

# Documentação de arquitetura – Visão estática

- Existem várias formas de documentar a arquitetura, contudo nesta abordagem a documentação é feita em UML, que é o padrão de modelagem do projeto orientado a objeto. Segundo Bass, Clements e Kazman (2010), são três as visões arquiteturais:
- **Visão modular:** representa a visão do sistema em termos de unidade de implementação; essas unidades podem ser classes, componentes ou módulos.
- **Visão componente e conector:** representa a forma pela qual os componentes interagem, ou seja, seus protocolos de comunicação.
- **Visão de alocação:** representa a forma pela qual esses componentes estão distribuídos em uma infraestrutura.

| Visão Arquitetural          | Diagrama UML                                             |
|-----------------------------|----------------------------------------------------------|
| Visão modular               | Diagrama de pacotes<br>Diagrama de classes               |
| Visão componente e conector | Diagrama de componentes                                  |
| Visão de alocação           | Diagrama de implantação<br>(ou Diagrama de distribuição) |

# Documentação de arquitetura – Visão dinâmica

O principal diagrama que representa a visão dinâmica da arquitetura é o diagrama de sequência.

Um conjunto de diagramas da UML servem como complemento ao diagrama de sequência. Os principais diagramas que dão suporte ao diagrama de sequência são:

- **Diagrama de colaboração (ou Diagrama de comunicação)** – as mensagens são as mesmas do diagrama de sequência, porém não se preocupa com a temporalidade do processo, concentra-se apenas nas mensagens trocadas entre os objetos.
- **Diagrama de máquinas de estado** – este diagrama, para ser construído, se baseia no diagrama de classes, ou diagrama de casos de uso. Este diagrama se preocupa com os métodos e mensagens dos objetos, úteis para representar o diagrama de sequência.



# Interatividade

Leia as definições abaixo no que se refere aos diagramas da UML:

- I. Este diagrama apresenta atributos, operações e relacionamentos entre os objetos.
- II. Este diagrama apresenta somente o atributo e os valores em uma determinada situação.
- III. Este diagrama é uma unidade de *software* que pode ser reusada em outros sistemas.

- |    |                  |                   |                    |
|----|------------------|-------------------|--------------------|
| a) | I. – classe;     | II. – objeto;     | III. – componente. |
| b) | I. – classe;     | II. – mensagem;   | III. – componente. |
| c) | I. – componente; | II. – mensagem;   | III. – estado.     |
| d) | I. – objeto;     | II. – componente; | III. – estado.     |
| e) | I. – objeto;     | II. – componente; | III. – classe.     |

# Resposta

Leia as definições abaixo no que se refere aos diagramas da UML:

- I. Este diagrama apresenta atributos, operações e relacionamentos entre os objetos.
- II. Este diagrama apresenta somente o atributo e os valores em uma determinada situação.
- III. Este diagrama é uma unidade de *software* que pode ser reusada em outros sistemas.

- |    |                  |                   |                    |
|----|------------------|-------------------|--------------------|
| a) | I. – classe;     | II. – objeto;     | III. – componente. |
| b) | I. – classe;     | II. – mensagem;   | III. – componente. |
| c) | I. – componente; | II. – mensagem;   | III. – estado.     |
| d) | I. – objeto;     | II. – componente; | III. – estado.     |
| e) | I. – objeto;     | II. – componente; | III. – classe.     |



# Referências

- ALBIN, S. T. *The art of software architecture: design methods and techniques*. Indianapolis: John Wiley, 2003.
- BASS, L.; CLEMENTS, P.; KAZMAN, R. *Software architecture in practice*. Boston: Addison-Wesley, 2010.
- BUSCHMANN, F. *et al. Pattern-oriented software architecture: a system of patterns*. New York: Wiley, 1996. v. 1
- ELMASRI, R.; NAVATHE, S. B. *Sistemas de banco de dados*. 6. ed. São Paulo: Addison-Wesley, 2011.
  - INTERNATIONAL ORGANIZATION FOR STANDARDIZATION (ISO). *ISO 25010: systems and software engineering – Systems and software quality requirements and evaluation (square) – system and software quality models*. Geneve: ISO, 2011a.

# Referências

- INTERNATIONAL ORGANIZATION FOR STANDARDIZATION (ISO). *ISO 42010: systems and software engineering – Architecture description*. Geneve: ISO, 2011b.
- OPENUP. *Introduction to OpenUP*. [s.d.]a. Disponível em: <[http://epf.eclipse.org/wikis/openup/publish.openup.base/guidances/supportingmaterials/introduction\\_to\\_openup\\_EFA29EF3.html?nodeId=e073de25](http://epf.eclipse.org/wikis/openup/publish.openup.base/guidances/supportingmaterials/introduction_to_openup_EFA29EF3.html?nodeId=e073de25)>. Acesso em: 4 maio 2015.
- PRESSMAN, R. S. *Engenharia de software*. 6. ed. São Paulo: McGraw-Hill, 2006.
- SOFTWARE ENGINEERING INSTITUTE (SEI). *Architecture trade off analysis method*, 2015a. Disponível em: . Acesso em: 4 maio 2015.
  - SILBERSCHATZ, A.; KORTH, H. F.; SUDARSHAN, S. *Sistema de banco de dados*. 5. ed. Rio de Janeiro: Campus, 2006.
  - SOMMERVILLE, I. *Engenharia de software*. São Paulo: Pearson, 2010.

**ATÉ A PRÓXIMA!**